ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Project: Anomaly Detection Systems

Task: Build a market anomaly detection platform for fraud and manipulation identification. Implement statistical and ML-based anomaly scoring, alert prioritization, and investigation workflows. Create pattern recognition for unusual trading activity and false positive management.

CONFIDENTIALITY & INTELLECTUAL PROPERTY NOTICE
STRICT CONFIDENTIALITY REQUIREMENTS All work is STRICTLY PRIVATE & CONFIDENTIAL

Mandatory Requirements:
• ALL GitHub repositories MUST be set to PRIVATE
• NO public sharing of code, concepts, or implementations
• ALL work remains property of Zetheta Algorithms Private Limited
• NDA compliance required throughout project lifecycle

GitHub Repository Requirements:
• Create as PRIVATE repository from Day 1
• Add ONLY authorized collaborators

• Enable branch protection rules
• Transfer ownership to @ZethetaIntern on Day 15

RAPID DEVELOPMENT NOTICE

This project MUST be completed in 15 days using AI/LLM tools

• Timeline: 15 days from start to deployment
• Approach: 80% AI-generated code, 20% customization
• Tools: Claude.ai, ChatGPT, Cursor, Scikit-learn, PyOD
• Success Metric: Live anomaly detection system

PROJECT OVERVIEW
Project Name: Anomaly Detection Systems
Project Duration: 15 Days
Difficulty Level: Intermediate-Advanced

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Executive Summary

Anomaly Detection Systems is a fraud and risk detection platform where students build systems that identify unusual trading activity, fraudulent transactions, and market anomalies. Players develop detection algorithms, implement scoring systems, and manage alert systems. The platform teaches anomaly detection techniques, fraud prevention, and the operational challenges of real-time monitoring.

Core Learning Objectives

- Understand anomaly detection techniques
- Master fraud detection algorithms
- Learn scoring methodologies
- Build detection pipelines
- Implement alert systems
- Understand false positive management
- Develop monitoring frameworks
- Create analytics systems

Game Concept & Storyline

You architect an anomaly detection system protecting against fraud and manipulation. Identify unusual trading patterns, detect fraudulent activity, and generate alerts. Balance sensitivity and false positive rate to maximize real threats caught.

Core Game Mechanics

- Data collection
- Feature engineering
- Model development
- Anomaly scoring
- Alert generation
- Investigation workflows
- Performance optimization
- Continuous improvement

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Gamification Elements

• Achievement badges for anomaly discoveries
• XP for correct detections
• Daily detection challenges
• Leaderboards by detection rate
• False positive minimization penalties
• Model accuracy rewards

PART 1: COMPREHENSIVE TRAINING MATERIAL

1.1 CORE CONCEPTS TO MASTER

Anomaly Detection Fundamentals

Supervised Methods:
• Classification models
• Logistic regression
• Random forests
• Gradient boosting

Unsupervised Methods:
• Clustering (K-means, DBSCAN)
• Isolation Forest
• Local Outlier Factor
• One-class SVM

Semi-Supervised:
• Anomaly detection with labels
• Label propagation
• Self-training
• Active learning

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Statistical Methods

Distribution-Based:
• Gaussian distribution
• Z-score detection
• Modified Z-score
• Mahalanobis distance

Time Series Methods:
• Moving average deviation
• Seasonal decomposition
• ARIMA residuals
• Exponential smoothing

Ensemble Methods:
• Multiple algorithms
• Voting schemes
• Weighted combinations
• Stacking approaches

Fraud Detection

Transaction Fraud:
• Unusual amounts
• Geographic anomalies
• Temporal patterns
• Velocity checks

Account Fraud:
• Access patterns
• Device changes
• Behavioral shifts
• Peer comparison

Trading Fraud:
• Front-running detection
• Spoofing detection
• Layering detection
• Manipulation patterns

Scoring Systems

Risk Scoring:
• Feature weighting

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

• Score calibration
• Threshold selection
• Confidence intervals

Behavioral Scoring:
• User profiling
• Baseline establishment
• Deviation measurement
• Personalization

Composite Scoring:
• Multi-model fusion
• Expert weighting
• Dynamic thresholds
• Contextual adjustment

Alert Management

Alert Generation:
• Rule-based alerts
• Model-based alerts
• Threshold-based alerts
• Escalation rules

Alert Quality:
• False positive rates
• False negative rates
• Alert precision
• Alert recall

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Alert Handling:
• Investigation workflows
• Case management
• Resolution tracking
• Feedback loops

False Positive Management

Root Cause Analysis:
• Model understanding
• Feature importance
• Decision trees
• Human review

Reduction Techniques:
• Feature engineering
• Model refinement
• Threshold tuning
• Context incorporation

Human-in-the-Loop:
• Analyst feedback
• Model updates
• Retraining triggers
• Continuous learning

Drift & Adaptation

Concept Drift:
• Data distribution changes
• Fraud pattern evolution
• Seasonal effects
• Regime changes

Adaptation Strategies:
• Retraining schedules
• Incremental learning
• Ensemble updates
• Model selection

Performance Monitoring

KPIs & Metrics:
• Detection rate

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

• False positive rate
• Precision/recall
• ROC-AUC

Monitoring System:
• Real-time dashboards
• Alert volume tracking
- Investigation efficiency
- Model performance

1.2 ESSENTIAL LEARNING RESOURCES
YouTube: Anomaly detection, Fraud detection, ML pipelines
Online: Coursera, edX, Kaggle competitions
Technical: Scikit-learn, PyOD, XGBoost, Isolation Forest

1.3 HANDS-ON LABS
- Feature engineering pipeline
- Model trainer
- Alert system implementation

1.4 REFERENCES
- Anomaly detection papers
- Fraud prevention research

PART 2: IMPLEMENTATION GUIDE
2.1 Development: 15-day sprint
2.2 Architecture: Python + ML framework
2.3 Stack: Scikit-learn, PyOD, XGBoost, FastAPI, PostgreSQL
2.4 Infrastructure: AWS with real-time data
2.5 Database: Transactions, anomalies, alerts, investigations
2.6 Tools: Cursor, Jupyter, GitHub
2.7 Testing: Unit, Integration, Evaluation tests

PART 3: CODE ARCHITECTURE
3.1 Data Pipeline:
- Data collection
- Feature engineering

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Normalization
- Quality checks

3.2 Detection Engine:
- Model training
- Anomaly scoring
- Threshold application
- Alert generation

3.3 Alert System:
- Alert routing
- Investigation workflow
- Feedback collection
- Model updates

PART 4: SUCCESS CRITERIA
Metrics: Detection rate >85%, False positives <5%, Uptime 99.5%+

Day 15: System live, alerts working, GitHub transfer
RESOURCES & NOTES
Key Websites:
• PyOD: https://pyod.readthedocs.io/
• Scikit-learn: https://scikit-learn.org/
• Kaggle: https://www.kaggle.com/

Tools:
• Pandas: https://pandas.pydata.org/
• Jupyter: https://jupyter.org/

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Important Notes:
• Validate detection accuracy
• Monitor false positive rates
• Test on real-world scenarios
• Maintain strict confidentiality

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Project: Anomaly Detection Systems
Training and Implementation Guide

PROJECT OVERVIEW
Project Name: Anomaly Detection Systems
Project Duration: 15 Days
Difficulty Level: Intermediate-Advanced

Executive Summary

Anomaly Detection Systems is a comprehensive fraud and risk detection platform where students build enterprise-grade systems that identify unusual trading activity, fraudulent transactions, and market anomalies in real-time. Participants develop sophisticated detection algorithms, implement multi-layered scoring systems, and manage complex alert workflows. The platform teaches cutting-edge anomaly detection techniques, fraud prevention methodologies, and the operational challenges of real-time monitoring at scale.This project provides hands-on experience with machine learning pipelines, statistical anomaly detection, behavioral analytics, and the practical challenges of building systems that balance detection accuracy with operational efficiency. Students learn to manage false positives, adapt to evolving fraud patterns, and build systems that can handle millions of transactions while maintaining low latency.

Core Learning Objectives

Technical Skills:
• Master statistical and machine learning anomaly detection techniques
• Understand supervised, unsupervised, and semi-supervised approaches
• Learn feature engineering for fraud detection
• Build real-time detection pipelines
• Implement ensemble methods for improved accuracy
• Develop adaptive learning systems
• Create scalable data processing architectures
• Build efficient alert management systems

Analytical Skills:
• Analyze fraud patterns and behaviors
• Evaluate model performance with appropriate metrics
• Balance precision and recall in detection systems

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

• Understand concept drift and adaptation strategies
• Perform root cause analysis on false positives
• Design scoring systems with business context
• Optimize threshold selection for various use cases
• Create investigation workflows

Business Skills:
• Understand fraud economics and impact
• Design alert prioritization schemes
• Manage operational efficiency
• Balance security and user experience
• Create feedback loops with investigations
• Communicate technical concepts to stakeholders
• Design compliance-aware systems
• Measure ROI of fraud prevention

Game Concept & Storyline

You are the Chief Security Officer at a major financial institution processing millions of transactions daily. Your mission is to architect and deploy an advanced anomaly detection system that protects against fraud, market manipulation, and insider trading while maintaining a seamless experience for legitimate users.The challenge is multifaceted: sophisticated fraudsters constantly evolve their techniques, normal behavior patterns shift over time, and every false positive costs investigation time and potentially damages customer relationships. You must build systems that are accurate, fast, adaptive, and explainable.As threats emerge and patterns evolve, you'll need to continuously refine your detection algorithms, incorporate new data sources, and optimize your alert workflows. Success means catching real threats while minimizing disruption to legitimate activity.

ADD EXPERIENCE TO KNOWLEDGE

Core Game Mechanics

Data Collection & Processing:
• Multi-source data ingestion
• Real-time streaming processing
• Historical data analysis
• Feature extraction and transformation
• Data quality monitoring

• Schema evolution handling
• Privacy-preserving techniques
• Scalable storage strategies

Feature Engineering:
• Transaction features (amount, frequency, timing)
• Behavioral features (patterns, deviations)
• Network features (relationships, connections)
• Temporal features (time-based patterns)
• Contextual features (location, device, channel)
• Derived features (ratios, aggregations)
• Interaction features (combinations)
• Domain-specific features

Model Development:
• Algorithm selection and comparison
• Hyperparameter optimization
• Ensemble model construction
• Model validation and testing
• Performance benchmarking
• Explainability analysis
• Deployment preparation
• A/B testing frameworks

Anomaly Scoring:
• Multi-model score fusion
• Confidence calculation
• Risk level assignment
• Priority determination
• Threshold application
• Score calibration
• Contextual adjustment
• Dynamic scoring

Alert Generation:
• Rule-based triggers
• Model-based triggers

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Composite triggers
- Alert enrichment
- Priority assignment
- Routing logic
- Escalation rules
- Deduplication

Investigation Workflows:
- Case creation and assignment
- Evidence collection
- Analysis tools
- Decision support
- Resolution tracking
- Feedback capture
- Learning integration
- Performance metrics

Performance Optimization:
- Model retraining schedules
- Feature selection refinement
- Threshold optimization
- Processing efficiency
- Latency reduction
- Resource utilization
- Cost optimization
- Scalability improvements

Continuous Improvement:
- Drift detection
- Adaptation triggers
- Incremental learning
- Ensemble updates
- Feature evolution
- Metric monitoring
- Feedback incorporation
- System evolution

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Gamification Elements

Achievement System:
• "First Blood" - First successful fraud detection
• "Eagle Eye" - Detect complex multi-step fraud
• "Speed Demon" - Catch fraud within 1 minute
• "Pattern Master" - Identify new fraud pattern
• "Clean Sweep" - Week with zero false positives
• "Guardian" - Prevent $1M+ in fraud losses
• "Detective" - Investigate 100 cases
• "Innovator" - Implement new detection technique

XP & Progression:
• +100 XP per confirmed fraud detection
• +50 XP per investigation completed
• +200 XP per new pattern identified
• +25 XP per model improvement
• -50 XP per false positive (incentive for precision)
• Bonus XP for detection speed
• Level up unlocks advanced techniques

Daily Challenges:
• "Rush Hour" - Monitor high-volume trading periods
• "Pattern Hunt" - Identify specific fraud types
• "Optimization Day" - Reduce false positive rate
• "Speed Trial" - Minimize detection latency
• "Threshold Tuning" - Optimize alert precision
• "Feature Discovery" - Create new features
• "Model Marathon" - Test multiple algorithms

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Leaderboards:
• Detection rate rankings
• False positive minimization
• Investigation efficiency
• Model accuracy scores
• Total fraud prevented
• Response time leaders

• Innovation awards

Penalties & Incentives:
• False positive penalties (investigation cost)
• Missed fraud penalties (loss amount)
• Late detection penalties (time-based)
• Accuracy bonuses
• Efficiency rewards
• Innovation bonuses
• Consistency rewards

PART 1: COMPREHENSIVE TRAINING MATERIAL

1.1 FUNDAMENTAL CONCEPTS - ANOMALY DETECTION THEORY

Introduction to Anomaly Detection

Anomaly detection, also called outlier detection, is the identification of rare items, events, or observations that differ significantly from the majority of data. In financial contexts, anomalies often indicate fraud, errors, or unusual market conditions that require investigation.The fundamental premise of anomaly detection is that normal instances are far more common than anomalies, and anomalies differ significantly from normal patterns. This assumption holds well in fraud detection where fraudulent transactions typically represent less than 1% of all transactions.

Types of Anomalies

Understanding the different types of anomalies is crucial for selecting appropriate detection techniques:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Point Anomalies:
Individual data points that deviate from normal patterns. These are the simplest and most common type of anomaly. Examples include:

- A single transaction for $50,000 when typical transactions are $50-100
- A purchase from a location 5,000 miles from user's home
- A transaction at 3 AM for a user who typically transacts during business hours

- An unusually large number of failed login attempts

Point anomalies are relatively straightforward to detect using statistical methods like Z-score analysis or distance-based approaches. The challenge lies in defining "normal" in a way that captures legitimate variation while flagging true anomalies.

Detection approaches:
- Statistical tests (Z-score, modified Z-score, Grubbs' test)
- Distance-based methods (K-nearest neighbors, LOF)
- Density-based methods (DBSCAN, LOF)
- Model-based methods (isolation forest, one-class SVM)

Contextual Anomalies (Conditional Anomalies):
Data points that are anomalous in a specific context but normal in others. Context can be defined by time, location, user attributes, or any other relevant feature. Examples:

- A $5,000 purchase is normal for a business account but anomalous for a student account
- Trading activity at 3 AM is normal for international traders but suspicious for domestic only accounts
- Multiple transactions in quick succession might be normal during holiday shopping but suspicious otherwise
- High-value purchases are normal for VIP customers but warrant scrutiny for new accounts

The key challenge with contextual anomalies is defining and capturing the relevant context. This requires domain knowledge and careful feature engineering to represent context appropriately.

Detection approaches:
- Conditional probability models
- Context-specific baselines
- Hierarchical models (segment data by context)
- Feature engineering to encode context
- Personalized anomaly detection

Collective Anomalies (Group Anomalies):
Collections of data points that together represent anomalous behavior, though individual points may appear normal. These are the most challenging to detect and often indicate sophisticated fraud schemes. Examples:

- A series of small withdrawals from different ATMs within an hour - each individually normal but collectively indicating card theft
- Coordinated trading activity across multiple accounts suggesting market manipulation
- Gradual increases in transaction amounts testing fraud detection limits
- Multiple accounts sharing similar behavioral patterns suggesting identity theft ring

Collective anomalies require sequence analysis, pattern recognition, and often graph-based techniques to identify relationships between instances.

Detection approaches:
- Sequence mining algorithms
- Time series analysis (ARIMA, LSTM)
- Graph analysis (community detection, centrality measures)
- Association rule mining
- Hidden Markov Models

Supervised vs. Unsupervised vs. Semi-Supervised Approaches

The choice between supervised, unsupervised, and semi-supervised approaches depends on data availability, problem characteristics, and operational constraints.

Supervised Anomaly Detection

Treats the problem as a classification task: normal versus anomalous. Requires labeled training data with known examples of both normal and fraudulent instances.

Advantages:
- High accuracy when labeled data is representative of future patterns
- Clear performance metrics (precision, recall, F1-score, ROC-AUC)
- Interpretable decision boundaries especially with tree-based methods
- Can capture complex patterns through ensemble methods
- Direct optimization of business objectives through cost-sensitive learning
- Well-established evaluation frameworks
- Supports online learning and model updates

Challenges:

- Requires significant labeled data (thousands of examples minimum)
- Labeling is expensive and time-consuming
- Labels may be delayed (fraud discovered weeks later)
- Extreme class imbalance (fraud is rare, typically <1%)
- May not detect novel fraud patterns not present in training data
- Vulnerable to adversarial adaptation (fraudsters evolve techniques)
- Risk of overfitting to historical patterns
- Concept drift requires frequent retraining

Common Algorithms:
1. Logistic Regression: Simple baseline, interpretable coefficients
2. Decision Trees: Interpretable rules, handles non-linear patterns
3. Random Forests: Robust ensemble, feature importance, reduced overfitting
4. Gradient Boosting (XGBoost, LightGBM): State-of-the-art performance on tabular data
5. Neural Networks: Captures complex patterns, requires more data
6. Support Vector Machines: Effective with proper kernel selection

Best Use Cases:
- When labeled fraud examples are available
- For known fraud patterns that repeat
- When interpretability is important
- For real-time scoring with low latency requirements
- When business objectives can be encoded as loss functions

Unsupervised Anomaly Detection

Identifies anomalies without labeled data by finding patterns that deviate from normal behavior. Assumes that anomalies are rare and significantly different from normal data.

ADD EXPERIENCE TO KNOWLEDGE

Advantages:
- No labeling required, reducing cost and time
- Can detect novel anomaly types not seen before
- Adapts to changing normal behavior automatically
- Useful for exploratory analysis and hypothesis generation
- Works when fraud labels are unavailable or unreliable
- Can identify previously unknown fraud schemes
- Natural fit for anomaly detection problem framing

Challenges:
- May produce many false positives
- Difficult to validate without labels
- Threshold selection is subjective and requires domain expertise
- May miss subtle anomalies similar to normal behavior
- Performance varies significantly by algorithm and parameters
- Hard to optimize for business objectives
- Requires careful feature engineering
- Sensitive to noisy data and outliers

Common Algorithms:
1. Statistical Methods: Z-score, IQR, Mahalanobis distance
2. Clustering: K-means, DBSCAN, Gaussian Mixture Models
3. Density-Based: Local Outlier Factor (LOF)
4. Isolation-Based: Isolation Forest
5. Dimensionality Reduction: PCA, Autoencoders
6. Distance-Based: K-nearest neighbors
7. One-Class SVM: Learns boundary around normal data

Best Use Cases:
- When labeled data is unavailable
- For discovering new fraud patterns
- During exploratory analysis
- For complementing supervised methods
- When fraud evolves rapidly
- For detecting zero-day attacks

Semi-Supervised Anomaly Detection

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Hybrid approach using small amounts of labeled data combined with large amounts of unlabeled data. Typically learns from normal examples and identifies deviations as anomalies.

Advantages:
- Leverages abundant unlabeled data
- Requires fewer labeled examples than fully supervised
- Better generalization than unsupervised alone

- Can detect novel anomalies while learning from examples
- More practical for many real-world scenarios
- Balances cost and performance

Challenges:
- Still requires some labeled data
- Quality of normal examples is critical
- May struggle if normal behavior is diverse
- Model selection and validation is complex
- Hyperparameter tuning is challenging

Common Techniques:
1. One-Class SVM: Learns boundary around normal data
2. Autoencoders: Train on normal data, high reconstruction error indicates anomaly
3. Semi-supervised GAN: Generator creates normal-like data
4. Pseudo-labeling: Iteratively labels confident predictions
5. Co-training: Multiple views of data train separate models
6. Self-training: Model labels its own confident predictions

Best Use Cases:
- When some labeled normal data exists
- For cold-start scenarios with new products
- When fraud patterns change frequently
- For combining benefits of supervised and unsupervised

Practical Considerations

Data Quality:
Anomaly detection is highly sensitive to data quality. Missing values, errors, inconsistencies, and noise can all be flagged as anomalies or mask real anomalies.

Key data quality practices:
- Implement data validation at ingestion
- Handle missing values appropriately (imputation vs. flagging)
- Detect and correct errors systematically
- Monitor data distributions over time
- Maintain data lineage and provenance

- Version datasets for reproducibility

Feature Engineering:
The quality of features often matters more than algorithm selection. Good features capture relevant patterns while being robust to noise.

Principles of good features:
- Domain-relevant: Capture known fraud indicators
- Discriminative: Differ between fraud and legitimate
- Robust: Stable across time and contexts
- Interpretable: Understandable to investigators
- Efficient: Fast to compute for real-time use
- Diverse: Capture different aspects of behavior

Evaluation Challenges:
Standard accuracy is misleading with imbalanced data. A model predicting "not fraud" for everything achieves 99%+ accuracy but catches zero fraud.

Appropriate metrics:
- Precision: Of flagged transactions, what percentage are fraud?
- Recall: Of actual fraud, what percentage do we detect?
- F1-Score: Harmonic mean balancing precision and recall
- PR-AUC: Performance across all thresholds (better than ROC-AUC for imbalanced data)
- Cost-based metrics: Expected value considering fraud losses and investigation costs
- Business metrics: Total fraud prevented, false decline rate, investigation efficiency

1.2 SUPERVISED ANOMALY DETECTION METHODS
Classification-Based Approaches

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

The supervised approach frames anomaly detection as binary classification: normal versus anomalous. This enables use of powerful classification algorithms and provides clear performance metrics.

Logistic Regression

Despite its simplicity, logistic regression often serves as a strong baseline and provides interpretable results through coefficient analysis.

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Mathematical Foundation:
The model estimates the probability of fraud as:
$P(Fraud|X) = 1 / (1 + \exp(-(\beta_0 + \beta_1 X_1 + \beta_2 X_2 + ... + \beta_n X_n)))$

Where:
- X represents feature vector
- β represents learned coefficients
- β_0 is the intercept term

Features with large positive coefficients strongly indicate fraud, while large negative coefficients indicate legitimate activity. This interpretability is valuable for regulatory compliance and investigator understanding.

Training Process:
1. Initialize coefficients (typically to zero)
2. For each training example:
   - Compute predicted probability
   - Calculate error (difference from actual label)
   - Update coefficients using gradient descent
3. Repeat until convergence or maximum iterations

Cost function minimized:
$J(\beta) = -1/m * \Sigma[y*\log(h(x)) + (1-y)*\log(1-h(x))] + \lambda*\Sigma(\beta^2)$

Where the first term is binary cross-entropy loss and the second is L2 regularization.

Advantages:
- Fast training and prediction (milliseconds for millions of samples)
- Interpretable coefficients show feature importance
- Probabilistic outputs enable threshold tuning
- Works well with linearly separable patterns
- Low computational and memory requirements
- Stable and robust to outliers in features
- Well-understood statistical properties

Limitations:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Assumes linear decision boundary
- Struggles with complex feature interactions
- May underfit complex fraud patterns
- Sensitive to feature scaling
- Requires careful handling of categorical variables

Implementation Best Practices:
- Scale features to similar ranges (standardization or normalization)
- Handle class imbalance using:
  * Class weights proportional to inverse frequency
  * SMOTE or other oversampling techniques
  * Threshold adjustment post-training
- Use regularization (L1 for feature selection, L2 for stability, or Elastic Net)
- Consider polynomial features for non-linear patterns
- Calibrate probabilities using Platt scaling or isotonic regression
- Monitor coefficient stability over time

Feature Engineering for Logistic Regression:
Since logistic regression assumes linear relationships, feature engineering is crucial:
- Create interaction terms: amount * velocity, location * time
- Add polynomial terms: $amount^2$, log(amount)
- Bin continuous variables: age_group, amount_bin
- Encode categorical variables: one-hot encoding, target encoding
- Create ratio features: amount/user_avg, velocity/user_avg

Decision Trees and Random Forests

Tree-based methods recursively partition the feature space to separate normal and anomalous instances. Random forests combine multiple trees for improved accuracy and robustness.

Decision Tree Mechanics:
A decision tree makes predictions by traversing from root to leaf based on feature values:

1. Start at root node containing all training data
2. Select feature and threshold that best splits data (maximizes information gain)
3. Create two child nodes based on split
4. Recursively repeat for each child until stopping criteria met

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

5. Assign class label to leaf nodes based on majority

Splitting Criteria:
- Gini Impurity: Measures class mixture in node
  Gini = 1 - $\Sigma(p^2)$ where p is proportion of each class

- Information Gain: Measures entropy reduction
  Gain = Entropy(parent) - Σ(weight * Entropy(child))
  Entropy = -Σ(p * log(p))

- Best split minimizes weighted impurity of children

Stopping Criteria:
- Maximum depth reached
- Minimum samples per leaf
- Minimum impurity decrease
- Perfect class separation achieved

Advantages of Decision Trees:
- Handle non-linear relationships naturally
- Automatic feature interaction detection
- No feature scaling required
- Can handle mixed data types (numerical and categorical)
- Interpretable through visualization
- Fast prediction after training
- Provide feature importance scores

Limitations of Decision Trees:
- Prone to overfitting (high variance)
- Unstable (small data changes cause large tree changes)
- Biased toward features with many values
- Cannot extrapolate beyond training data
- May create overly complex trees

Random Forests

Random forests address decision tree limitations through ensemble learning:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Algorithm:
1. Create N bootstrap samples of training data (sampling with replacement)
2. For each bootstrap sample:
   - Train decision tree
   - At each split, consider random subset of features (typically √n features)
   - Grow tree to maximum depth (no pruning)
3. Combine predictions:
   - Classification: Majority vote
   - Probabilities: Average predicted probabilities

Key Parameters:
- n_estimators: Number of trees (typically 100-500)
- max_depth: Maximum tree depth (controls overfitting)
- min_samples_split: Minimum samples to split node (2-10)
- min_samples_leaf: Minimum samples in leaf (1-4)
- max_features: Features considered per split (sqrt(n) or log2(n))
- class_weight: Handle imbalanced data ('balanced' or custom)

Advantages:
- Significantly more accurate than single trees
- Reduced overfitting through averaging
- Robust to outliers and noise
- Handles high-dimensional data well
- Provides confidence estimates
- Natural handling of missing values
- Excellent out-of-the-box performance

Feature Importance:
Random forests provide feature importance scores showing predictive power:

Mean Decrease in Impurity:
- For each feature, sum impurity decreases across all trees
- Normalize by number of trees
- Higher values indicate more important features

Permutation Importance:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Randomly permute feature values
- Measure decrease in model accuracy
- More reliable but computationally expensive

Best Practices:
- Use 100-500 trees (more is usually better but diminishing returns)
- Tune max_depth to prevent overfitting (5-20 typically)
- Use balanced class weights for fraud detection
- Monitor out-of-bag error during training
- Extract and analyze important decision rules
- Consider feature importance for feature selection
- Use stratified sampling for very imbalanced data

Gradient Boosting Machines

Gradient boosting builds an ensemble sequentially, where each tree corrects errors of previous trees. Modern implementations (XGBoost, LightGBM, CatBoost) provide exceptional performance.

Core Concept:
Instead of training trees independently like random forests, gradient boosting trains trees sequentially. Each new tree predicts the errors (residuals) of the existing ensemble.

Algorithm:
1. Initialize model with constant prediction (log-odds for classification)
2. For m = 1 to M (number of trees):
   a. Compute pseudo-residuals (gradients of loss function)
   b. Fit regression tree to pseudo-residuals
   c. Update ensemble: F_m(x) = F_(m-1)(x) + learning_rate * tree_m(x)
3. Final prediction: Apply logistic function to ensemble output

Mathematical Formulation:
$F(x) = F_0(x) + \Sigma(\eta * h_m(x))$

Where:
- F_0 is initial prediction
- η is learning rate (shrinkage factor)

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

- h_m is m-th tree
- Each tree minimizes: L = Σ(loss(y, F_(m-1)(x) + h_m(x)))

XGBoost (Extreme Gradient Boosting)

Key innovations making XGBoost extremely effective:

1. Regularized Objective:
   Obj = Σ(loss(y, pred)) + Σ(Ω(tree))

   Where Ω(tree) = γ*T + (λ/2)*Σ(w^2)
   - γ: Penalty for number of leaves
   - λ: L2 regularization on leaf weights
   - Prevents overfitting

2. Second-Order Optimization:
   - Uses second-order gradients (Hessian)
   - More accurate than first-order methods
   - Faster convergence

3. Efficient Implementation:
   - Parallel tree construction
   - Cache-aware access patterns
   - Sparsity-aware algorithms
   - Handles missing values automatically

Key Parameters:
- learning_rate (eta): 0.01-0.3 (smaller = more trees needed but better generalization)
- max_depth: 3-10 (deeper trees capture more interactions but risk overfitting)
- min_child_weight: Minimum sum of instance weight in child (prevents overfitting)
- subsample: 0.5-1.0 (fraction of samples used per tree)
- colsample_bytree: 0.5-1.0 (fraction of features used per tree)
- scale_pos_weight: Handles class imbalance (ratio of negative to positive)
- gamma: Minimum loss reduction for split (regularization)
- reg_alpha: L1 regularization
- reg_lambda: L2 regularization

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Advantages:
- State-of-the-art performance on structured/tabular data
- Handles missing values automatically (learns best direction)
- Built-in cross-validation
- Feature importance and SHAP values
- Efficient parallel processing
- Supports custom objectives and metrics
- Robust regularization prevents overfitting

LightGBM

Microsoft's gradient boosting framework optimized for speed and memory:

Key Innovations:
1. Histogram-Based Learning:
   - Bins continuous features into discrete buckets
   - Dramatically reduces memory and computation
   - Enables training on very large datasets

2. Leaf-Wise Growth:
   - Grows tree by best leaf (deepest) rather than level-wise
   - Can achieve better accuracy with fewer leaves
   - Requires careful max_depth setting to prevent overfitting

3. Categorical Feature Support:
   - Native handling without one-hot encoding
   - Finds optimal split points for categorical variables
   - Reduces dimensionality and improves performance

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Advantages:
- 10-20x faster than XGBoost on large datasets
- Lower memory usage
- Better accuracy with fewer iterations
- Excellent for high-dimensional data
- Superior handling of categorical features

CatBoost

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Yandex's gradient boosting library with focus on categorical features:

Key Innovations:
1. Ordered Boosting:
   - Addresses prediction shift problem
   - More robust to overfitting
   - Better generalization

2. Categorical Feature Handling:
   - Automatic encoding using target statistics
   - Handles high-cardinality categoricals
   - Prevents target leakage

3. Symmetric Trees:
   - Balanced tree structure
   - Faster prediction
   - Better regularization

Advantages:
- Best-in-class categorical feature handling
- Less parameter tuning required
- Robust to overfitting
- Good default parameters
- Fast prediction speed

Gradient Boosting Best Practices:

For Fraud Detection:
- Start with learning_rate=0.1, max_depth=6
- Use scale_pos_weight to handle imbalance
- Enable early stopping with validation set
- Monitor both training and validation metrics
- Use 100-1000 iterations (early stopping prevents overfitting)
- Cross-validate to select hyperparameters
- Consider cost-sensitive learning if fraud costs known

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Feature Engineering:
- Create interaction features (boosting will amplify)
- Include derived features (ratios, differences)
- Use temporal features (time since last transaction)
- Encode categorical variables appropriately
- Create aggregate features (user averages, counts)

Hyperparameter Tuning:
- Use grid search or random search for initial exploration
- Use Bayesian optimization for fine-tuning
- Focus on learning_rate, max_depth, min_child_weight first
- Then tune subsample and colsample_bytree
- Finally adjust regularization parameters
- Always use validation set or cross-validation

Model Monitoring:
- Track feature importance over time
- Monitor prediction distributions
- Check for overfitting (train vs validation gap)
- Validate on recent data regularly
- Retrain when performance degrades

Neural Networks for Fraud Detection

Deep learning approaches can capture highly complex patterns in high-dimensional data. While traditionally requiring more data and computation, they excel with large datasets and complex feature interactions.

Feed-Forward Neural Networks

ADD EXPERIENCE TO KNOWLEDGE

Architecture:
- Input layer: One node per feature
- Hidden layers: Dense layers with non-linear activations
- Output layer: Single node with sigmoid activation

Typical Architecture for Fraud Detection:
Input (features) → Dense(128, ReLU) → Dropout(0.3) → Dense(64, ReLU) → Dropout(0.3) →

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Dense(32, ReLU) → Dense(1, Sigmoid) → Output (probability)

Activation Functions:
- ReLU (Rectified Linear Unit): f(x) = max(0, x)
  * Most common for hidden layers
  * Prevents vanishing gradients
  * Fast computation

- Sigmoid: f(x) = 1 / (1 + e^-x)
  * Used for output layer (binary classification)
  * Outputs probability between 0 and 1

- Tanh: f(x) = (e^x - e^-x) / (e^x + e^-x)
  * Alternative for hidden layers
  * Outputs between -1 and 1

Training Process:
1. Forward propagation: Compute predictions
2. Compute loss (binary cross-entropy)
3. Backward propagation: Compute gradients
4. Update weights using optimizer (Adam, RMSprop)
5. Repeat for multiple epochs

Regularization Techniques:
- Dropout: Randomly disable neurons during training (0.2-0.5)
- L2 Regularization: Penalize large weights
- Early Stopping: Stop when validation loss stops improving
- Batch Normalization: Normalize activations between layers

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Key Considerations:
- Feature scaling is critical (standardize or normalize)
- Use batch normalization for stable training
- Monitor training and validation loss for overfitting
- Handle class imbalance through:
  * Class weights in loss function
  * Oversampling minority class
  * Custom loss functions

Strictly Private and Confidential – Not for Circulation

- Use appropriate learning rate (0.001 typically)
- Implement early stopping to prevent overfitting

Advantages:
- Captures highly complex patterns
- Automatic feature interaction learning
- Scalable to very large datasets
- Transfer learning possible
- Can incorporate different data types

Limitations:
- Requires more data than tree-based methods
- Longer training time
- Less interpretable (black box)
- More hyperparameters to tune
- Sensitive to feature scaling
- May overfit with small datasets

Recurrent Neural Networks (RNN/LSTM/GRU)

For sequential transaction data, recurrent architectures model temporal dependencies and patterns over time.

LSTM (Long Short-Term Memory):
Designed to capture long-term dependencies through gating mechanisms:

- Forget Gate: Decides what information to discard
- Input Gate: Decides what new information to store
- Output Gate: Decides what to output

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Architecture for Transaction Sequences:
Input Sequence → LSTM(64) → LSTM(32) → Dense(16) → Dense(1, Sigmoid) → Output

Use Cases:
- Detect patterns like "small test transaction followed by large withdrawal"
- Model user behavior sequences
- Identify gradually escalating fraud

- Capture temporal context (time of day, day of week patterns)

GRU (Gated Recurrent Unit):
Simpler than LSTM with fewer parameters:
- Combines forget and input gates into update gate
- Faster training with similar performance
- Good choice when compute is limited

Bidirectional RNNs:
Process sequences forward and backward to capture full context:
- Particularly useful when entire sequence is available
- Better understanding of patterns
- Slower but more accurate

Best Practices:
- Normalize sequences (transaction amounts, times)
- Pad sequences to equal length
- Use attention mechanisms to highlight important transactions
- Implement sequence truncation for very long histories
- Monitor for vanishing/exploding gradients

Attention Mechanisms and Transformers

Attention allows models to focus on relevant parts of input:

Self-Attention:
- Computes importance of each transaction relative to others
- Captures long-range dependencies efficiently
- More parallelizable than RNNs

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Transformer Architecture for Fraud:
- Embed transactions (amount, merchant, time, etc.)
- Multi-head self-attention captures different patterns
- Feed-forward layers process attended representations
- Classification head outputs fraud probability

Advantages:

- Captures complex temporal patterns
- Provides interpretability through attention weights
- Parallel processing (faster than RNNs)
- Scales well to long sequences

Use Cases:
- Analyzing complete transaction histories
- Understanding multi-step fraud schemes
- Identifying coordinated account activity
- Detecting behavioral changes over time

Handling Class Imbalance

Fraud detection faces extreme class imbalance - typically less than 1% of transactions are fraudulent. Standard classification algorithms optimize accuracy, which is misleading when classes are imbalanced.

Impact of Imbalance:
A model predicting "not fraud" for everything achieves 99%+ accuracy but catches zero fraud. This demonstrates why accuracy is an inappropriate metric for fraud detection.

Approaches to Handle Imbalance:

1. Sampling Techniques

Random Under-Sampling:
- Randomly remove normal instances to balance classes
- Pros: Fast, reduces training time, simple implementation
- Cons: Discards potentially useful information, may lose important patterns
- When to use: When normal instances are abundant (millions) and representative

Example: From 99,000 legitimate and 1,000 fraud transactions, randomly sample 1,000 legitimate to create balanced 1,000:1,000 dataset.

Random Over-Sampling:
- Replicate minority class instances to balance classes
- Pros: No information loss, simple implementation

- Cons: Risk of overfitting to repeated examples, increases training time
- When to use: When fraud examples are well-understood and diverse

SMOTE (Synthetic Minority Over-sampling Technique):
- Generate synthetic minority class examples
- Creates new instances by interpolating between existing fraud examples
- Algorithm:
  1. For each fraud instance, find k nearest fraud neighbors
  2. Randomly select one neighbor
  3. Create synthetic instance between them: new = instance + λ * (neighbor - instance)
     where λ is random between 0 and 1

- Pros: Creates diverse new examples, reduces overfitting, increases model generalization
- Cons: May create unrealistic examples, can blur class boundaries
- Variations: ADASYN (adaptive synthetic sampling), BorderlineSMOTE, SMOTE-ENN

Implementation Considerations:
- Apply SMOTE only to training data, never test data
- Combine with under-sampling (SMOTE-Tomek, SMOTE-ENN)
- Validate on original imbalanced data
- Monitor for unrealistic synthetic examples
- Consider domain constraints when generating synthetic data

2. Cost-Sensitive Learning

Assign different misclassification costs to different classes reflecting business reality.

Cost Matrix:

	Predicted Normal	Predicted Fraud
Actual Normal	0	C_FP
Actual Fraud	C_FN	0

Where:
- C_FP: Cost of false positive (investigation time, customer friction)
- C_FN: Cost of false negative (fraud loss amount)

In fraud detection, C_FN >> C_FP typically. Missing a $10,000 fraud is much costlier than investigating a $50 legitimate transaction.

Example: If average fraud loss is $1,000 and investigation costs $10, set fraud class weight to 100 times normal class weight.

Implementation:
- Most algorithms support class_weight parameter
- Sklearn: class_weight='balanced' or custom dictionary
- XGBoost: scale_pos_weight parameter
- Neural networks: Custom loss function with class weights

class_weight = {0: 1, 1: C_FN / C_FP}

3. Threshold Adjustment

Instead of default 0.5 threshold for classification, optimize based on business objectives.

ROC Curve (Receiver Operating Characteristic):
- Plots True Positive Rate vs False Positive Rate at various thresholds
- Shows trade-off between catching fraud (TPR) and false alarms (FPR)
- AUC (Area Under Curve) measures overall performance

Precision-Recall Curve:
- More informative for imbalanced data
- Plots Precision vs Recall at various thresholds
- Shows trade-off between alert accuracy and fraud detection

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Threshold Selection Strategies:

Maximize F1-Score:
- Balances precision and recall
- Good general-purpose metric
- F1 = 2 * (Precision * Recall) / (Precision + Recall)

Maximize Business Value:

- Minimize: Cost = FN * Cost_FN + FP * Cost_FP
- Optimal threshold depends on cost ratio
- Update as costs change

Operational Constraints:
- Investigation capacity: Can only handle N alerts per day
- Set threshold to generate N highest-risk alerts
- Dynamically adjust as capacity changes

4. Ensemble Methods

Build multiple models on different balanced subsets and combine predictions.

Balanced Random Forest:
- Each tree trained on balanced bootstrap sample
- Combines under-sampling with ensemble learning
- Maintains diversity while handling imbalance

Easy Ensemble:
- Create multiple balanced subsets through under-sampling
- Train classifier on each subset
- Combine with majority voting or averaging
- Leverages full dataset while maintaining balance

Advantages:
- Robust to sampling randomness
- Better generalization
- Captures different aspects of data

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

5. Anomaly Detection Framing

Instead of classification, frame as anomaly detection:
- Train on normal data only
- Detect fraud as deviations from normal
- Natural fit for fraud detection problem
- Handles evolving fraud patterns better

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Methods:
- One-class SVM
- Isolation Forest
- Autoencoders
- Robust covariance

Evaluation Metrics for Imbalanced Data

Never use accuracy alone. Always use multiple metrics:

Confusion Matrix:

	Predicted Normal	Predicted Fraud
Actual Normal	TN	FP
Actual Fraud	FN	TP

Precision (Positive Predictive Value):
- Precision = TP / (TP + FP)
- Of alerts generated, what percentage are true fraud?
- Important when investigation capacity is limited
- Higher precision means fewer wasted investigations

Recall (Sensitivity, True Positive Rate):
- Recall = TP / (TP + FN)
- Of actual fraud cases, what percentage do we detect?
- Important for risk management and fraud prevention
- Higher recall means more fraud caught

F1-Score:
- F1 = 2 * (Precision * Recall) / (Precision + Recall)
- Harmonic mean balancing precision and recall
- Good single metric for model comparison
- Gives equal weight to precision and recall

F-Beta Score:
- $F_\beta = (1 + \beta^2) * (\text{Precision} * \text{Recall}) / (\beta^2 * \text{Precision} + \text{Recall})$
- $\beta > 1$ weighs recall higher (catch more fraud)
- $\beta < 1$ weighs precision higher (fewer false positives)

Strictly Private and Confidential – Not for Circulation

- Choose β based on business priorities

ROC-AUC (Receiver Operating Characteristic - Area Under Curve):
- Measures overall discrimination ability
- Value between 0.5 (random) and 1.0 (perfect)
- 0.9+ is excellent, 0.8-0.9 is good, 0.7-0.8 is acceptable
- Less informative for highly imbalanced data

PR-AUC (Precision-Recall - Area Under Curve):
- More informative than ROC-AUC for imbalanced data
- Focuses on positive class (fraud) performance
- Always use for fraud detection evaluation
- Higher is better, no fixed interpretation

Cost-Based Metrics:
- Expected Cost = FN * Cost_FN + FP * Cost_FP + TP * (Cost_Investigation - Value_Prevented)
- Expected Value = (TP * Value_Prevented) - (FP * Cost_FP) - (TP * Cost_Investigation)
- Most directly aligned with business objectives
- Requires knowing or estimating costs

Matthews Correlation Coefficient:
- MCC = (TP*TN - FP*FN) / sqrt((TP+FP)(TP+FN)(TN+FP)(TN+FN))
- Range: -1 to +1 (0 is random)
- Balanced measure even with imbalanced classes
- Good for model comparison

Best Practices:
- Report multiple metrics (precision, recall, F1, PR-AUC)
- Include confusion matrix for full understanding
- Show performance at different thresholds
- Calculate metrics on realistic test set (original imbalance)
- Monitor metrics over time for drift detection
- Segment analysis (by transaction type, amount, user segment)

1.3 UNSUPERVISED ANOMALY DETECTION METHODS
Clustering-Based Approaches

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Clustering algorithms group similar instances together. Instances that don't fit well into any cluster or are far from cluster centers are potential anomalies.

K-Means Clustering

Partitions data into K clusters by minimizing within-cluster variance.

Algorithm:
1. Initialize K cluster centers:
   - Random selection
   - K-means++ (choose centers far apart)
   - Domain-based initialization

2. Iterate until convergence:
   a. Assignment step: Assign each point to nearest center
      $cluster(i) = argmin_k \|x_i - center_k\|^2$

   b. Update step: Recalculate centers as mean of assigned points
      $center_k = (1/|C_k|) * \Sigma(x_i \text{ for } x_i \text{ in } C_k)$

3. Convergence when centers don't change or maximum iterations reached

Anomaly Detection with K-Means:
- Calculate distance of each point to nearest cluster center
- Points with distance exceeding threshold are anomalies
- Anomaly score = distance to nearest center
- Can normalize by cluster radius for consistency

Distance Metrics:
- Euclidean: Most common, assumes spherical clusters
- Manhattan: Less sensitive to outliers
- Cosine: For high-dimensional sparse data
- Mahalanobis: Accounts for feature correlations

Advantages:
- Simple and intuitive

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Fast computation O(n*K*i*d) where i is iterations, d is dimensions
- Scales to large datasets
- Easy to interpret cluster assignments
- Works well for spherical clusters

Limitations:
- Requires specifying K (number of clusters)
- Assumes clusters are spherical and similar size
- Sensitive to initialization (use K-means++)
- Struggles with varying density clusters
- Outliers affect cluster centers
- Assumes features have similar scales

Choosing K:
- Elbow method: Plot within-cluster sum of squares vs K, look for "elbow"
- Silhouette analysis: Measures how similar points are to their own cluster vs others
- Domain knowledge: K based on known user segments
- Cross-validation: Try multiple K values, validate on holdout set

Implementation Tips:
- Always use K-means++ initialization
- Normalize features to [0,1] or standardize
- Run multiple times with different initializations
- Consider mini-batch K-means for very large datasets
- Use K-medoids for outlier resistance
- Validate clusters make business sense

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Groups together points that are closely packed while marking points in low-density regions as anomalies (noise). Unlike K-means, doesn't force every point into a cluster.

Key Concepts:
- Eps (ε): Neighborhood radius
- MinPts: Minimum points within Eps to form dense region

Point Types:

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

1. Core Point: Has at least MinPts neighbors within Eps
2. Border Point: Within Eps of core point but has fewer than MinPts neighbors
3. Noise Point: Neither core nor border - potential anomalies!

Algorithm:
1. For each unvisited point p:
   a. Mark as visited
   b. Find all points within Eps (neighborhood)
   c. If neighborhood has < MinPts points:
      - Mark p as noise (anomaly)
   d. If neighborhood has ≥ MinPts points:
      - Start new cluster with p
      - Add all reachable points to cluster (density-connected)
2. Continue until all points visited

Density-Reachable:
Point q is density-reachable from p if there's a chain of core points from p to q, each within Eps of the next.

Anomaly Detection with DBSCAN:
- Points marked as noise are anomalies
- No threshold needed - algorithm identifies anomalies directly
- Natural for fraud detection: fraudsters are "noise" in normal behavior clusters

Advantages:
- Detects clusters of arbitrary shape
- Automatically identifies anomalies (noise points)
- No need to specify number of clusters
- Robust to outliers (doesn't let them influence clusters)
- Handles varying cluster densities reasonably well
- Intuitive parameters with physical meaning

Limitations:
- Sensitive to Eps and MinPts parameter selection
- Struggles with varying density clusters (different Eps needed)
- High-dimensional data challenges (curse of dimensionality)
- Computational complexity $O(n^2)$ without spatial indexing

Strictly Private and Confidential – Not for Circulation

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

- Border points can be assigned to different clusters (order-dependent)

Parameter Selection:
For Eps:
- Plot k-distance graph (distance to k-th nearest neighbor)
- Look for "elbow" point where distance increases sharply
- Start with k = MinPts

For MinPts:
- Rule of thumb: MinPts ≥ dimensions + 1
- Start with 2 * dimensions
- Increase for noisy data, decrease for sparse data
- Domain knowledge about minimum cluster size

Implementation Tips:
- Use spatial indexing (KD-trees, Ball trees) for efficiency O(n log n)
- Apply dimensionality reduction for high-dimensional data (PCA, t-SNE)
- Consider HDBSCAN for hierarchical density-based clustering
- Normalize features to ensure Eps is meaningful
- Visualize clusters to validate parameters
- Try multiple parameter combinations

Variations:
- HDBSCAN: Hierarchical DBSCAN, less parameter-sensitive
- OPTICS: Orders points by reachability, creates dendrogram
- ST-DBSCAN: Spatiotemporal extension for time-series data

Gaussian Mixture Models (GMM)

Models data as a mixture of multiple Gaussian (normal) distributions. Provides probabilistic cluster assignments and natural anomaly scores.

Mathematical Foundation:
$P(x) = \Sigma(\pi_k * N(x|\mu_k, \Sigma_k))$ for k = 1 to K

Where:
- π_k: Weight of k-th Gaussian (mixture coefficient), $\Sigma\pi_k = 1$

Strictly Private and Confidential – Not for Circulation

- μ_k: Mean vector of k-th Gaussian
- Σ_k: Covariance matrix of k-th Gaussian
- $N(x|\mu,\Sigma)$: Multivariate Gaussian density

Each Gaussian represents a cluster. Points are assigned probabilistically to clusters based on likelihood.

Training (Expectation-Maximization Algorithm):
1. Initialize parameters (π_k, μ_k, Σ_k) randomly or with K-means
2. E-step: Calculate probability each point belongs to each cluster
   $\gamma_{ik} = P(\text{cluster } k \mid x_i) = [\pi_k * N(x_i|\mu_k,\Sigma_k)] / [\Sigma_j \pi_j * N(x_i|\mu_j,\Sigma_j)]$
3. M-step: Update parameters based on probabilities
   $\pi_k = (1/n) * \Sigma_i \gamma_{ik}$
   $\mu_k = \Sigma_i(\gamma_{ik} * x_i) / \Sigma_i \gamma_{ik}$
   $\Sigma_k = \Sigma_i(\gamma_{ik} * (x_i-\mu_k)(x_i-\mu_k)^T) / \Sigma_i \gamma_{ik}$
4. Repeat until convergence (log-likelihood stops increasing)

Anomaly Detection with GMM:
- Calculate P(x) for each instance
- Low probability indicates anomaly
- Anomaly score = -log(P(x))
- Set threshold on probability or score

Covariance Types:
- Full: Each cluster has full covariance matrix (flexible, many parameters)
- Diagonal: Features are independent within clusters (fewer parameters)
- Spherical: Single variance per cluster (simplest, most constrained)
- Tied: All clusters share same covariance structure

ADD EXPERIENCE TO KNOWLEDGE

Advantages:
- Soft cluster assignments (probabilistic)
- Natural anomaly scores (probabilities)
- Handles elliptical clusters well
- Provides uncertainty estimates
- Interpretable as mixture of distributions
- Can model complex distributions

Limitations:
- Requires specifying number of components K
- Sensitive to initialization
- Can converge to local optima
- Computationally expensive for large K
- Assumes Gaussian distributions
- May require many components for complex distributions

Model Selection:
- BIC (Bayesian Information Criterion): Penalizes model complexity
  BIC = -2 * log-likelihood + K * log(n)
  Lower is better

- AIC (Akaike Information Criterion): Alternative information criterion
  AIC = -2 * log-likelihood + 2 * K
  Lower is better

- Cross-validation: Evaluate likelihood on holdout set

Implementation Tips:
- Initialize with K-means for stability
- Try multiple initializations, keep best (highest likelihood)
- Start with diagonal covariance, try full if needed
- Use regularization for numerical stability
- Consider Bayesian GMM for automatic component selection
- Validate clusters are meaningful

Distance-Based Methods

These methods identify anomalies based on distance to other points or to learned representations.

Isolation Forest

Explicitly designed for anomaly detection based on the principle that anomalies are easier to isolate than normal points. This counterintuitive approach is highly effective and efficient.

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Core Insight:
Anomalies are:
1. Few in number (rare)
2. Different from normal points (feature values unusual)

Therefore, anomalies require fewer random splits to isolate than normal points!

Algorithm:
1. Build Isolation Trees (iTrees):
   a. Select random feature
   b. Select random split value between min and max of that feature
   c. Recursively partition data by creating binary tree
   d. Stop when:
      - Point is isolated (alone in partition)
      - Tree reaches maximum depth
      - All points in node are identical

2. Repeat to create forest of iTrees (typically 100-200)

3. For anomaly detection:
   a. For each point, calculate average path length across all trees
   b. Path length = number of edges from root to leaf containing point
   c. Shorter average path length → easier to isolate → more anomalous

Anomaly Score:
$s(x, n) = 2^{(-E(h(x)) / c(n))}$

Where:
- $E(h(x))$: Average path length for instance x across all trees
- $c(n)$: Average path length of unsuccessful search in BST (normalization)
- n: Number of instances

Score interpretation:
- $s \approx 1$: Anomaly
- $s \approx 0.5$: Normal
- $s < 0.5$: Very normal

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Advantages:
- Specifically designed for anomaly detection
- Low computational complexity: O(n log n)
- Handles high-dimensional data well
- No distance calculations required
- Works well without parameter tuning
- Effective for both global and local anomalies
- Memory efficient through subsampling
- Naturally handles irrelevant features

Limitations:
- May struggle with clustered anomalies (anomalies similar to each other)
- Performance depends on contamination parameter (expected anomaly fraction)
- Less interpretable than distance-based methods (why is path short?)
- Subsampling may miss rare patterns in very large datasets

Parameters:
- n_estimators: Number of trees (100-200 typical, more is better but diminishing returns)
- max_samples: Subsample size (256 common, trades speed vs thoroughness)
- contamination: Expected proportion of anomalies (0.01-0.1 typical)
- max_features: Features to consider per split (1.0 = all features)

Implementation Tips:
- No feature scaling needed (split points random regardless of scale)
- Works out-of-the-box with default parameters
- Set contamination based on domain knowledge of fraud rate
- Use max_samples=256 for speed on large datasets (>100k instances)
- Consider feature importance for interpretation (though less direct than trees)
- Combine with other methods for robust detection
- Monitor score distributions to set thresholds

Use Cases:
- Primary detector when labeled data unavailable
- Complement to supervised methods (ensemble)
- High-dimensional fraud detection
- Real-time scoring (fast prediction)
- Exploratory analysis of unusual patterns

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Local Outlier Factor (LOF)

Measures local density deviation of a point compared to its neighbors. Points in sparse regions relative to neighbors are flagged as anomalies. Excellent for detecting local anomalies that might be normal globally.

Key Concept:
A point is anomalous if its local density is significantly lower than its neighbors' densities. "Local" is key - compares point to neighborhood, not global dataset.

Definitions:
1. k-distance(A): Distance from point A to its k-th nearest neighbor

2. Reachability Distance (RD):
   RD_k(A,B) = max(k-distance(B), distance(A,B))

   Why max? Ensures statistical stability by setting minimum distance based on B's neighborhood. If A is very close to B, still uses B's k-distance.

3. Local Reachability Density (LRD):
   LRD_k(A) = 1 / [average RD_k(A, neighbor) for all k-nearest neighbors]

   Inverse of average reachability distance to neighbors. Higher LRD means point is in dense region.

4. Local Outlier Factor (LOF):
   LOF_k(A) = [average LRD_k(neighbor) / LRD_k(A)] for all k-nearest neighbors

   Ratio of neighbors' densities to point's density.

LOF Interpretation:
- LOF ≈ 1: Similar density to neighbors (normal)
- LOF >> 1: Much lower density than neighbors (anomaly)
- LOF < 1: Higher density than neighbors (deep inside cluster)

Typical thresholds: LOF > 1.5 or 2.0 for anomalies

Strictly Private and Confidential – Not for Circulation

Advantages:
- Detects local anomalies effectively
- Works with varying density clusters (major advantage over global methods)
- Considers local neighborhood structure
- Provides interpretable anomaly scores
- No assumption about global data distribution
- Handles clusters of different densities

Limitations:
- Computationally expensive: $O(n^2)$ for large datasets
- Sensitive to k parameter (neighborhood size)
- Struggles with high-dimensional data (curse of dimensionality)
- Memory intensive for large datasets
- Doesn't handle new data naturally (must recompute for prediction)

Parameter Selection:
k (number of neighbors):
- Small k (10-20): Sensitive to local patterns, may overfit
- Large k (50-100): More stable, may miss subtle local patterns
- Rule of thumb: k = 2 * dimensions
- Try multiple values, use ensemble or majority voting
- Depends on dataset size (larger datasets can use larger k)

Implementation Tips:
- Use approximate nearest neighbor methods for large datasets (Annoy, FAISS)
- Apply dimensionality reduction for high dimensions (PCA, t-SNE, UMAP)
- Consider novelty detection variant (LOF fitted on training, predict on new data)
- Normalize features to ensure distance metrics are meaningful
- Use spatial indexing (KD-trees, Ball trees) to speed up
- For streaming data, use micro-cluster based approach
- Combine with Isolation Forest for comprehensive coverage

Variations:
- COF (Connectivity-Based Outlier Factor): Uses graph connectivity
- LOCI (Local Correlation Integral): Parameter-free variant
- LoOP (Local Outlier Probabilities): Probabilistic LOF

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

- Simplified LOF: Uses k-nearest neighbor distance directly

One-Class SVM

Learns a decision boundary around normal data in feature space. Points outside this boundary are classified as anomalies. Uses kernel trick to handle non-linear boundaries in high-dimensional spaces.

Concept:
Train on normal data only, learn boundary that contains most normal points while minimizing boundary size (margin maximization).

Mathematical Formulation:
Find hyperplane w that maximizes distance from origin while containing most data:

$\min (1/2)||w||^2 + (1/(nu*n)) * \Sigma\xi_i - \rho$

Subject to: $w \cdot \varphi(x_i) \geq \rho - \xi_i$ and $\xi_i \geq 0$

Where:
- $\varphi(x)$: Kernel function mapping to high-dimensional space
- ρ: Offset (distance of hyperplane from origin)
- ξ_i: Slack variables (allow some points outside boundary)
- nu: Upper bound on fraction of outliers, lower bound on fraction of support vectors

The nu parameter controls the trade-off between boundary tightness and flexibility.

Kernel Functions:
1. RBF (Radial Basis Function) - Most common:
   $K(x,y) = \exp(-gamma * ||x-y||^2)$
   - gamma: Controls decision boundary smoothness
   - High gamma: Tight, wiggly boundary (may overfit)
   - Low gamma: Smooth, loose boundary (may underfit)

2. Linear:
   $K(x,y) = x \cdot y$
   - Use for high-dimensional data

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Fast computation
- Interpretable

3. Polynomial:
   $K(x,y) = (\text{gamma} * x \cdot y + \text{coef}_0)^{\text{degree}}$
   - Captures polynomial interactions
   - Rarely used for anomaly detection

4. Sigmoid:
   $K(x,y) = \tanh(\text{gamma} * x \cdot y + \text{coef}_0)$
   - Neural network inspired
   - Rarely used in practice

Advantages:
- Theoretically well-founded (margin maximization)
- Effective in high dimensions through kernels
- Robust decision boundary
- Works with small training samples (support vectors)
- Good generalization with proper regularization
- Can handle non-linear patterns

Limitations:
- Computationally intensive: $O(n^2)$ to $O(n^3)$
- Sensitive to kernel and parameter choice
- Difficult to interpret (kernel space transformations)
- Requires careful feature scaling
- Memory intensive for large datasets
- Training time increases significantly with data size

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Parameters:
- nu: Expected fraction of anomalies (0.01-0.5)
  * Controls boundary tightness
  * Set based on expected fraud rate
  * Too small: Overly tight boundary, high false negatives
  * Too large: Loose boundary, high false positives

- kernel: Transformation function

  * 'rbf': Good default, handles non-linear patterns
  * 'linear': Use for high-dimensional or linearly separable data

- gamma: Kernel coefficient for RBF
  * 'scale': 1 / (n_features * X.var()) - good default
  * 'auto': 1 / n_features
  * Higher values: More complex boundaries
  * Grid search for optimal value

Implementation Tips:
- Always scale features to [0,1] or standardize (critical for SVM)
- Use RBF kernel as default, try linear for high dimensions (>1000 features)
- Set nu to expected fraud rate (e.g., 0.05 for 5% fraud)
- For large datasets (>10k samples):
  * Use SGDOneClassSVM (stochastic gradient descent variant)
  * Or subsample data for training
- Grid search for gamma: [0.0001, 0.001, 0.01, 0.1, 1]
- Cross-validate on normal data using reconstruction error
- Consider Isolation Forest for large datasets (much faster)

Mahalanobis Distance

Measures distance accounting for correlations between features and variance in different directions. More sophisticated than Euclidean distance for multivariate data.

Motivation:
Euclidean distance treats all dimensions equally, but features often have:
- Different scales (amount in dollars vs. count)
- Different variances (some features vary more than others)
- Correlations (velocity and amount might correlate)

Mahalanobis distance accounts for these through covariance structure.

Mathematical Definition:
$D_M(x) = \text{sqrt}[(x - \mu)^T \Sigma^{-1} (x - \mu)]$

Where:

- x: Data point
- μ: Mean vector of distribution
- Σ: Covariance matrix
- Σ^{-1}: Inverse covariance matrix (precision matrix)

Squared distance follows chi-squared distribution with d degrees of freedom (d = dimensions):
$D_M^2(x) \sim \chi^2(d)$

This enables threshold selection using chi-squared distribution percentiles.

Properties:
1. Scale-invariant: Automatically accounts for different feature scales
2. Rotation-invariant: Distance unchanged by linear transformations
3. Equals Euclidean when features are uncorrelated with equal variance
4. Reduces to standardized Euclidean when features uncorrelated but different variance

Anomaly Detection:
1. Estimate μ and Σ from normal training data
2. For each test point, calculate D_M(x)
3. Points with D_M(x) > threshold are anomalies
4. Threshold from chi-squared distribution (e.g., 97.5th percentile for α=0.025)

Advantages:
- Accounts for feature correlations naturally
- Scale-invariant (no normalization needed)
- Statistical foundation for threshold selection
- Effective for multivariate Gaussian data
- Simple and interpretable

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Limitations:
- Assumes Gaussian distribution (may not hold for fraud data)
- Sensitive to covariance estimation:
  * Requires sufficient samples (>>dimensions)
  * Outliers in training data affect covariance
  * May be singular or ill-conditioned
- Breaks down with high-dimensional data (d approaching n)
- Global method: Doesn't capture local anomalies

- Computationally expensive for large covariance matrices

Robust Estimation:
Standard mean and covariance are sensitive to outliers. Use robust estimators:

Minimum Covariance Determinant (MCD):
- Finds subset of h points (e.g., 75% of data) with minimum covariance determinant
- Estimates μ and Σ from this subset
- More robust to outliers in training data
- Recommended for fraud detection

Implementation:
```
from sklearn.covariance import EmpiricalCovariance, MinCovDet

# Robust covariance estimation
mcd = MinCovDet().fit(X_normal)
mahal_dist = mcd.mahalanobis(X_test)

# Threshold using chi-squared distribution
from scipy.stats import chi2
threshold = chi2.ppf(0.975, df=X.shape[1])  # 97.5th percentile

anomalies = mahal_dist > threshold
```

Implementation Tips:
- Use robust covariance estimation (MCD) always
- Ensure training data is predominantly normal (contamination <10%)
- Requires n >> d (at least 5-10 times more samples than features)
- Apply dimensionality reduction if d is large (PCA)
- Features should be approximately Gaussian (transform if needed)
- Regularize covariance matrix for stability
- Validate Gaussian assumption (Q-Q plots, statistical tests)
- Consider as baseline or feature for ensemble

Feature Transformations:
Improve Gaussian assumption:
- Log transform: For right-skewed features (amounts)

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

- Box-Cox transform: General power transformation
- Quantile transform: Map to uniform or Gaussian distribution

1.4 STATISTICAL ANOMALY DETECTION METHODS

Distribution-Based Techniques

These methods assume normal data follows a known statistical distribution and identify points that deviate significantly from this distribution.

Z-Score Method

Measures how many standard deviations a data point is from the mean. Foundation of many statistical tests.

Formula:
$Z = (x - \mu) / \sigma$

Where:
- x: Data point
- μ: Population or sample mean
- σ: Population or sample standard deviation

Interpretation:
- $|Z| < 1$: Within 1 standard deviation (~68% of data)
- $|Z| < 2$: Within 2 standard deviations (~95% of data)
- $|Z| < 3$: Within 3 standard deviations (~99.7% of data)
- $|Z| > 3$: Extreme outlier (<0.3% of data if normally distributed)

Common thresholds:
- Conservative: $|Z| > 3$ (fewer false positives)
- Moderate: $|Z| > 2.5$
- Aggressive: $|Z| > 2$ (more sensitive)

Assumptions:
- Data is approximately normally distributed
- Mean and standard deviation are representative
- Single peak distribution (unimodal)

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- No significant outliers influencing μ and σ

Advantages:
- Simple to implement and understand
- Fast computation O(n)
- Clear statistical interpretation (probability)
- No training required
- Works well for Gaussian data
- Can be applied per feature independently

Limitations:
- Assumes Gaussian distribution (violated for fraud data)
- Sensitive to outliers in μ and σ calculation
- Only works for univariate or independent features
- Fixed threshold may not suit all contexts
- Doesn't account for feature correlations
- Breaks down with skewed distributions

Multi-Variate Extension:
For multiple features, can compute Z-score for each and combine:
- Maximum: $\max(|Z_1|, |Z_2|, ..., |Z_n|)$
- Sum: $\Sigma|Z_i|$
- Sum of squares: $\Sigma(Z_i^2)$
- Any exceeds threshold: flag if any $|Z_i| >$ threshold

Better: Use Mahalanobis distance which accounts for correlations

Implementation Best Practices:
- Check distribution before applying (histogram, Q-Q plot)
- Use modified Z-score or MAD-based methods for robustness
- Transform skewed features (log, Box-Cox)
- Apply per relevant subgroup (user segment, transaction type)
- Combine with domain rules
- Don't use for highly skewed or multi-modal data

Modified Z-Score (Robust Z-Score)

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Uses median and MAD (Median Absolute Deviation) instead of mean and standard deviation. More robust to outliers.

Formula:
M = 0.6745 * (x - median) / MAD

Where:
- MAD = median($|x_i$ - median$|$)
- 0.6745: Scaling factor making M comparable to Z-score under normality
  (0.6745 ≈ 0.75th quantile of standard normal)

Why More Robust:
- Median: 50% of data must change to shift it (vs. mean affected by single outlier)
- MAD: Measures spread using median rather than mean of deviations
- Both have breakdown point of 50% (can tolerate up to 50% contamination)

Threshold:
Typically |M| > 3.5 (slightly higher than Z-score threshold)

Advantages over Z-Score:
- Robust to outliers in baseline estimation
- Better for contaminated data (some anomalies in training)
- Maintains interpretability
- Still fast to compute
- Works with non-Gaussian distributions better

Use When:
- Training data may contain some anomalies
- Distribution has heavy tails
- Outliers significantly affect mean/std
- Skewed distributions

Interquartile Range (IQR) Method

Non-parametric method using quartiles to identify outliers. No distributional assumptions.

Definitions:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Q1 (First Quartile): 25th percentile
- Q2 (Second Quartile): 50th percentile (median)
- Q3 (Third Quartile): 75th percentile
- IQR = Q3 - Q1

Outlier Detection:
- Lower fence: Q1 - 1.5 * IQR
- Upper fence: Q3 + 1.5 * IQR
- Outliers: x < lower fence OR x > upper fence
- Extreme outliers: x < Q1 - 3*IQR OR x > Q3 + 3*IQR

Factor 1.5:
- Tukey's original choice
- For normal distribution, captures 99.3% of data
- Can adjust for sensitivity:
  * 1.5: Standard (moderate sensitivity)
  * 2.0: Conservative (fewer outliers)
  * 1.0: Aggressive (more outliers)

Advantages:
- No distributional assumptions (non-parametric)
- Robust to extreme outliers (uses percentiles)
- Simple and widely understood (box plot method)
- Works with skewed data
- Visual representation available (box plot)
- Fast computation

Limitations:
- Only captures extreme outliers (may miss subtle anomalies)
- No continuous anomaly score (binary: outlier or not)
- Treats all features independently
- May be too conservative or aggressive depending on data
- Doesn't adapt to local patterns
- Fixed threshold across all data

Best For:
- Exploratory data analysis

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

- Quick outlier detection
- Skewed or non-Gaussian data
- Interpretable results for stakeholders
- Identifying data quality issues

Grubbs' Test

Statistical hypothesis test for detecting a single outlier in univariate data assumed to be Gaussian.

Null Hypothesis (H_0):
No outliers in dataset

Alternative Hypothesis (H_1):
Exactly one outlier exists

Test Statistic:
$G = \max|x_i - \text{mean}| / \text{std}$

Steps:
1. Calculate mean and std of data
2. Find point most distant from mean
3. Calculate G statistic
4. Compare to critical value from Grubbs' distribution table
5. If G > critical value: Reject H_0 (outlier detected)
6. Remove outlier and repeat (iterative process)

Critical Values:
Depend on:
- Sample size n
- Significance level α (typically 0.05 or 0.01)
- Available in statistical tables or computed via:
  $G_critical = ((n-1)/\text{sqrt}(n)) * \text{sqrt}(t^2/(n-2+t^2))$
  where t is from t-distribution with n-2 degrees of freedom

Advantages:
- Formal statistical test with p-values
- Controls false positive rate (Type I error)

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Well-defined procedure
- Theoretical foundation

Limitations:
- Assumes Gaussian distribution (strong assumption)
- Tests only one outlier at a time (iterative for multiple)
- Masking: Multiple outliers hide each other
- Swamping: Non-outliers flagged due to outliers
- Not suitable for fraud detection (too slow, wrong assumptions)

Better Alternatives for Fraud:
- Use for data cleaning, not real-time detection
- Consider Dixon's Q test for very small samples
- Use robust methods (MAD-based) for practical applications

Time Series Anomaly Detection

Financial transaction data has temporal patterns requiring specialized techniques. Standard methods ignore temporal structure.

Moving Average Methods

Compare current values to moving averages to detect sudden changes. Adapt to trends while flagging deviations.

Simple Moving Average (SMA):
$MA_t = (1/w) * \Sigma x_{t-i}$ for i=0 to w-1

Where w is window size.

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Anomaly Detection:
- Calculate residual: $r_t = x_t - MA_t$
- Flag if $|r_t| > \text{threshold} * \text{std(residuals)}$

Advantages:
- Simple and intuitive
- Adapts to trends (moving baseline)

- Smooths noise
- Fast computation

Limitations:
- Lags behind actual values (delayed detection)
- Equal weight to all points in window
- Doesn't capture seasonality
- Window size is critical parameter

Exponential Moving Average (EMA):
$MA_t = \alpha * x_t + (1-\alpha) * MA_{t-1}$

Where $\alpha \in (0,1)$ is smoothing factor.
- Higher α: More weight to recent values (responsive)
- Lower α: More weight to history (smooth)

Advantages over SMA:
- More weight to recent data (faster adaptation)
- Single parameter (α vs. window size)
- Computationally efficient (recursive)
- Better for trending data

Anomaly Detection Strategies:
1. Control Chart Approach:
   - Upper control limit: $MA + k*\sigma$
   - Lower control limit: $MA - k*\sigma$
   - Flag points outside control limits
   - k typically 2-3

2. Adaptive Thresholding:
   - Threshold = $c * MA$
   - Flag if $x_t > threshold$
   - Useful for proportion-based anomalies

3. Rate of Change:
   - Flag if $|x_t - x_{t-1}| > threshold$
   - Detects sudden spikes or drops

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Best Practices:
- Choose window size based on pattern timescale
- Smaller window: Detects short-term anomalies
- Larger window: Smoother baseline, detects major shifts
- For multiple timescales, use multiple windows
- Combine with domain rules (e.g., weekend adjustments)

Seasonal Decomposition

Separates time series into components to isolate anomalies. Essential for data with regular patterns.

Components:
x(t) = Trend(t) + Seasonal(t) + Residual(t)

Or multiplicative:
x(t) = Trend(t) × Seasonal(t) × Residual(t)

Definitions:
- Trend: Long-term progression (growth, decline)
- Seasonal: Regular periodic patterns (daily, weekly, yearly)
- Residual: Irregular component (contains anomalies)

Decomposition Methods:

Classical Decomposition:
1. Calculate trend using moving average
2. Detrend: x_t - Trend_t
3. Calculate seasonal component (average by period)
4. Residual = Original - Trend - Seasonal

Limitations:
- Assumes constant seasonal pattern
- Sensitive to outliers
- Poor at series endpoints

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

STL (Seasonal-Trend decomposition using LOESS):
- Robust to outliers
- Handles changing seasonal patterns
- Flexible specification of smoothness
- Recommended for anomaly detection

Algorithm:
1. Initial trend estimate
2. Detrending
3. Cycle-subseries smoothing (seasonal)
4. Low-pass filtering (trend)
5. Iterate until convergence

Parameters:
- seasonal: Smoothness of seasonal component
- trend: Smoothness of trend component
- robust: Whether to use robust fitting (recommended: True)

Anomaly Detection After Decomposition:
1. Extract residual component
2. Apply statistical tests to residuals:
   - Z-score on residuals
   - IQR on residuals
   - Control charts
3. Flag large residuals as anomalies

Interpretation:
- Large positive residual: Higher than expected from trend and seasonality
- Large negative residual: Lower than expected
- Pattern in residuals: Model misspecification

Advantages:
- Removes confounding temporal patterns
- Reveals true anomalies vs. regular variation
- Interpretable components
- Handles multiple seasonalities

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Best Practices:
- Choose decomposition period matching dominant seasonality
- Use STL for robustness
- Validate seasonal pattern makes sense
- Handle multiple seasonalities (daily + weekly) with nested decomposition
- Check residuals are random (no patterns)

ARIMA-Based Detection

Fit autoregressive integrated moving average model to time series. Large prediction errors indicate anomalies.

ARIMA(p, d, q):
- p: Autoregressive order (lags of series)
- d: Differencing order (make stationary)
- q: Moving average order (lags of errors)

Model:
$(1 - \Sigma\varphi_i L^i) (1-L)^d y_t = (1 + \Sigma\theta_i L^i) \varepsilon_t$

Where:
- φ_i: AR coefficients
- θ_i: MA coefficients
- L: Lag operator
- ε_t: White noise

Process:
1. Check stationarity (ADF test, KPSS test)
2. If non-stationary, apply differencing
3. Identify p, q using ACF/PACF plots
4. Fit ARIMA model
5. Validate residuals are white noise
6. Generate predictions
7. Calculate residuals: $r_t = y_t - predicted_t$
8. Flag large $|r_t|$ as anomalies

Advantages:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

- Captures temporal dependencies
- Provides prediction intervals (confidence bands)
- Statistical foundation
- Flexible for various patterns
- Well-established theory

Limitations:
- Requires stationarity (or achievable through differencing)
- Model selection is complex (p, d, q)
- Computationally intensive for large data
- May overfit to historical patterns
- Assumes linear relationships
- Breaks down with concept drift

ARIMAX Extension:
Include exogenous variables:
- Day of week, holidays
- Promotional events
- External economic indicators
- Improves predictions and anomaly detection

Seasonal ARIMA (SARIMA):
ARIMA(p,d,q)(P,D,Q)m

Adds seasonal components:
- P, D, Q: Seasonal AR, differencing, MA orders
- m: Seasonal period (7 for daily with weekly seasonality)

Implementation Tips:
- Use auto_arima for automatic model selection
- Validate assumptions (stationarity, residuals)
- Update model regularly (weekly or monthly)
- Consider simpler methods for real-time applications
- Use for batch anomaly detection, not real-time
- Combine with domain knowledge

Prophet

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

Facebook's time series forecasting tool designed for business time series with strong seasonal effects and multiple seasonality levels.

Model:
$y(t) = g(t) + s(t) + h(t) + \varepsilon_t$

Components:
- g(t): Piecewise linear or logistic growth (trend)
- s(t): Periodic seasonal component (Fourier series)
- h(t): Holiday effects
- ε_t: Error term

Trend g(t):
Piecewise linear with automatic changepoint detection:
- Detects when growth rate changes
- Flexible to gradual and sudden changes
- User can specify known changepoints

Seasonality s(t):
Fourier series for periodic patterns:
$s(t) = \Sigma[a_n \cos(2\pi nt/P) + b_n \sin(2\pi nt/P)]$

Where P is period (365.25 for yearly, 7 for weekly)

Multiple seasonalities handled simultaneously:
- Yearly: Holiday patterns, annual cycles
- Weekly: Weekday vs weekend effects
- Daily: Hourly patterns

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

Holiday Effects h(t):
- User specifies holidays/events
- Model learns impact (positive or negative)
- Custom windows around events
- Country-specific calendars

Anomaly Detection with Prophet:

ZeTheta Algorithms Private Limited
CIN No. U62012MH2023PTC410415
Registered Office Address: F2307 Oberoi Splendor JVLR
Mumbai – 400060, Maharashtra, India.
Email: hr@zetheta.com
Phone: +91-9136249369

1. Fit model on historical data
2. Generate predictions with uncertainty intervals
3. Calculate residuals: actual - predicted
4. Flag points outside prediction intervals
5. Or flag large standardized residuals

Uncertainty Intervals:
- Trend uncertainty: From changepoint estimation
- Seasonal uncertainty: From parameter uncertainty
- Observation noise: Additional variation
- Combined into prediction intervals

Advantages:
- Handles missing data automatically
- Robust to outliers
- Multiple seasonalities
- Holiday effects
- Interpretable components
- Fast fitting
- Minimal tuning required

Best For:
- Daily or higher frequency data
- Multiple seasonal patterns
- Known holidays/events
- Business metrics (sales, usage)
- Situations requiring interpretability

Parameters:
- growth: 'linear' or 'logistic'
- changepoints: Dates of known trend changes
- seasonality_mode: 'additive' or 'multiplicative'
- yearly_seasonality: True/False or Fourier order
- weekly_seasonality: True/False or Fourier order
- daily_seasonality: True/False or Fourier order

Implementation:

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE

python
```
from fbprophet import Prophet

# Fit model
model = Prophet(yearly_seasonality=True, weekly_seasonality=True)
model.add_country_holidays(country_name='US')
model.fit(df)

# Predict
forecast = model.predict(df_future)

# Anomaly detection
df['residual'] = df['y'] - forecast['yhat']
df['anomaly'] = (df['y'] < forecast['yhat_lower']) | (df['y'] > forecast['yhat_upper'])
```
Exponential Smoothing (Holt-Winters)

State space approach for forecasting with level, trend, and seasonal components. Efficient for real-time anomaly detection.

Triple Exponential Smoothing:
- Level equation: $\ell_t = \alpha(y_t - s_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1})$
- Trend equation: $b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)b_{t-1}$
- Seasonal equation: $s_t = \gamma(y_t - \ell_t) + (1-\gamma)s_{t-m}$
- Forecast: $\hat{y}_{t+h} = \ell_t + h*b_t + s_{t+h-m}$

Where:
- α: Level smoothing parameter
- β: Trend smoothing parameter
- γ: Seasonal smoothing parameter
- m: Seasonal period

Parameters (all $\in [0,1]$):
- Higher values: More responsive (follow recent data)
- Lower values: More smooth (follow historical patterns)

Advantages:
- Computationally efficient (recursive updates)

- Handles level, trend, seasonality
- Quick adaptation to changes
- Real-time capable
- Simple parameter interpretation

Anomaly Detection:
1. Fit model on training data
2. Generate h-step ahead predictions
3. Calculate prediction intervals using residual variance
4. Flag points outside intervals

Best For:
- Real-time monitoring
- Short-term forecasting (1-10 steps)
- Clear seasonal patterns
- Operational dashboards

Limitations:
- Requires choosing smoothing parameters
- Fixed seasonal pattern
- Limited to uni or multiplicative models
- Less flexible than Prophet or ARIMA
- No automatic changepoint detection

Practical Usage:
python
```
from statsmodels.tsa.holtwinters import ExponentialSmoothing

# Fit model
model = ExponentialSmoothing(y, seasonal='add', seasonal_periods=7)
fit = model.fit()

# Forecast
forecast = fit.forecast(steps=7)

# Anomaly detection
residuals = y - fit.fittedvalues
```

```
threshold = 3 * residuals.std()
anomalies = abs(residuals) > threshold
```

ADDITIONAL INFORMATION

The following additional information can be extracted through LLMs/ Public websites to get a thorough and comprehensive material of learning for this project:

COMPREHENSIVE TRAINING MATERIAL

1.1 Fundamental Concepts - Anomaly Detection Theory
1.2 Supervised Anomaly Detection Methods
1.3 Unsupervised Anomaly Detection Methods
1.4 Statistical Anomaly Detection Methods
1.5 Fraud-Specific Detection Techniques
1.6 Scoring Systems and Risk Assessment
1.7 Alert Management and Operational Excellence
1.8 False Positive Management
1.9 Concept Drift and Model Adaptation
1.10 Advanced Topics and Emerging Techniques

IMPLEMENTATION GUIDE

2.1 Development Roadmap - 15-Day Sprint
2.2 Technical Architecture
2.3 Implementation Details with Code
2.4 Testing Strategy
2.5 Deployment Guide
3.1 Monitoring and Observability
3.2 Operational Procedures
3.3 Model Retraining Pipeline
4.1 Explainable AI Implementation
4.2 Real-Time Feature Store
4.3 Graph-Based Fraud Detection
5.1 Performance Metrics
5.2 Dashboard Examples
6.1 Learning Resources
7.1 Project Completion Checklist

ZeTheta
ADD EXPERIENCE TO KNOWLEDGE